

Table 1: The Linux process states

Name	Flag	Kernel-defined state name and description
Running	R	TASK_RUNNING: The process is either executing on a CPU or waiting to run. It can be executing user routines or kernel routines (system calls), or be queued and ready when in the running (or runnable) state.
Sleeping	S	TASK_INTERRUPTIBLE: The process is waiting for some condition — a hardware request, system resources access, or a signal. When an event or signal satisfies the condition, the process returns to the running (or runnable) state.
Sleeping	D	TASK_UNINTERRUPTIBLE: The process is waiting for some condition — a hardware request, system resources access, or a signal. When an event or signal satisfies the condition, the process returns to the running state.
Sleeping	K	TASK_KILLABLE: This is identical to the uninterruptible D state, but modified to allow the waiting task to respond to a signal to be killed (exited completely). The utilities frequency displays killable processes as the D state.
Stopped	T	TASK_STOPPED: The process has been stopped (suspended), usually by being signalled by a user or another process. The process can be continued (resumed) by another signal, in order to return to the running state.
	T	TASK_TRACED: A process that is being debugged is also temporarily stopped and shares the same T state flag.
Zombie	Z	EXIT_ZOMBIE: A child process signals its parent as it exits. All resources except for the process identity (PID) are released.
	X	EXIT_DEAD: When the parent cleans up (reaps) the remaining child process structure, the process is released completely. This state will never be observed in process listing utilities.

resource allocation change. Processes are assigned as states, which change with the circumstances. The Linux process states are explained in Table 1.

Listing processes

The `ps` command is used for listing current processes. The command can provide detailed process information including:

- The user identification (UID), which determines process privileges
- The unique process identification
- The CPU and real-time already expended
- How much memory the process has allocated in various locations
- The location of process STDOUT known as the controlling terminal
- The state of the current process

Important: The Linux version of `ps` supports three options/formats, including:

- UNIX (POSIX) options, which may be grouped and must be preceded by a dash
 - BSD options, which may be grouped and must not be used with a dash
 - GNU long options, which are preceded by two dashes
- For example, `ps -aux` is not the same as `ps aux`.

A common display listing (*options aux*) displays all processes, with columns that users will be interested in; this includes processes without a controlling terminal. A long listing (*options lax*) provides more technical details,

but may display faster by avoiding the user name lookup. A similar UNIX syntax uses the options `-ef` to display all processes. The output of some commands is shown in Figures 1, 2 and 3.

By default, `ps` with no option selects all the processes with the same effective user ID (EUID) as the current user, and is associated with the same terminal where `ps` was invoked.

- Processes in brackets (usually at the top) are scheduled kernel threads.
- `ps` displays once. The man page of `top` (1) can be referred to for a repetitive updated process display.

```

[root@server1 Desktop]# ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.7  0.3 52840 6608 ?        Ss   13:38   0:18 /usr/lib/systemd/systemd -
root           2  0.0  0.0      0     0 ?        S    13:38   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    13:38   0:00 [ksoftirqd/0]
root           5  0.0  0.0      0     0 ?        S<   13:38   0:00 [kworker/0:0H]
root           6  0.0  0.0      0     0 ?        S    13:38   0:00 [kworker/u4:0]
root           7  0.0  0.0      0     0 ?        S    13:38   0:01 [mlgation/0]
root           8  0.0  0.0      0     0 ?        S    13:38   0:00 [rcu_bh]
root           9  0.0  0.0      0     0 ?        S    13:38   0:00 [rcuob/0]
root          10  0.0  0.0      0     0 ?        S    13:38   0:00 [rcuob/1]
root          11  0.2  0.0      0     0 ?        S    13:38   0:05 [rcu_sched]
root          12  0.0  0.0      0     0 ?        S    13:38   0:01 [rcuos/0]
root          13  0.0  0.0      0     0 ?        S    13:38   0:01 [rcuos/1]
root          14  0.0  0.0      0     0 ?        S    13:38   0:00 [watchdog/0]
root          15  0.0  0.0      0     0 ?        S    13:38   0:00 [watchdog/1]
root          16  0.0  0.0      0     0 ?        S    13:38   0:01 [migration/1]
root          17  0.0  0.0      0     0 ?        S    13:38   0:00 [ksoftirqd/1]
root          19  0.0  0.0      0     0 ?        S<   13:38   0:00 [kworker/1:0H]
root          20  0.0  0.0      0     0 ?        S<   13:38   0:00 [khelper]
root          21  0.0  0.0      0     0 ?        S    13:38   0:00 [kdevtmpfs]
root          22  0.0  0.0      0     0 ?        S<   13:38   0:00 [netns]
root          23  0.0  0.0      0     0 ?        S<   13:38   0:00 [writeback]
root          24  0.0  0.0      0     0 ?        S<   13:38   0:00 [kintegrityd]
root          25  0.0  0.0      0     0 ?        S<   13:38   0:00 [bioset]
root          26  0.0  0.0      0     0 ?        S<   13:38   0:00 [kblockd]
root          27  0.0  0.0      0     0 ?        S    13:38   0:00 [khubd]
root          28  0.0  0.0      0     0 ?        S<   13:38   0:00 [md]
root          32  0.0  0.0      0     0 ?        S    13:38   0:00 [kswapd0]

```

Figure 1: The output of `ps aux`